



Course Github repo

DL 1 compressed

Prepared by Mishan Aliev



TG course chat

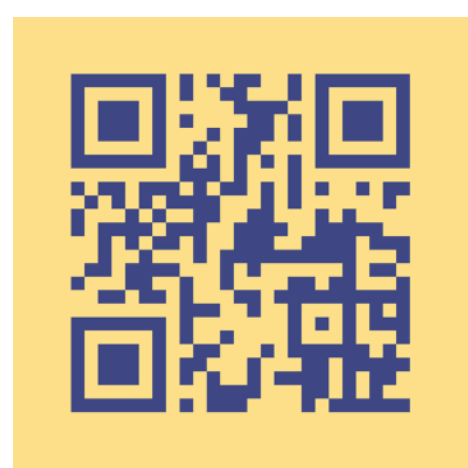
HSE University, 2026

Who am I?

My name is **Mishan Aliev**

- Junior Research Fellow, Center for Deep Learning and Bayesian Methods
- Lecturer, Department of Big Data and Information Retrieval, HSE University
- PhD student, HSE University × Yandex

My X



My LinkedIn



BayesLab seminar



Lecture Topics

- DL compressed
- Efficient Deep Learning essentials
- LLM 1
- LLM 2
- LLM 3
- CV 1
- CV2
- Generative models 1
- Generative models 2
- Generative models 3
- Generative models 4
- Generative models in 3D CV
- Deep Learning for Tabular Data
- Deep Learning for Graphs

Grading



Course Github repo

AI360 & MOP: $0.4 \cdot \text{practice} + 0.4 \cdot \text{hw} + 0.2 \cdot \text{final test}$

KNAD: $0.7 \cdot \text{hw} + 0.3 \cdot \text{final test}$

-
- 4 homeworks (1st one today 🤗)
 - Practice:
 - competition
 - task, which you have to transfer from another course
 - Final test: question from one lecture block

PART I

The deep learning pipeline

Mechanisms first; architectures later.

Today

One pipeline, then diagnostic cases

01

Compressed recap

data → objective → update

02

Questions and cases

hypothesis → diagnosis → test

03

Seminar

intervene → measure → explain

Roadmap

Follow one training pipeline from data to adaptation

01	Problem & data	examples · targets · split · transform
02	Tensors & batch	shape · dtype · device · padding · mask
03	Forward & objective	model · head · logits · loss · reduction
04	Backward & update	graph · gradients · optimizer · learning rate
05	Generalization & adaptation	regularization · evaluation · inference · fine-tuning

Basic recall

Deep learning in one equation

$$\hat{\theta} \approx \operatorname{argmin}_{\theta} \frac{1}{N} \sum_i l(f_{\theta}(x_i), y_i)$$

$f_{\theta}(\cdot)$ — model

$D, (x_i, y_i) \in D$ — dataset

$l(\cdot)$ — loss function (what counts as an error)

Optimizer searches for $\hat{\theta}$, but it doesn't promise a global optimum.

Basic recall

The data define the task before the model does

raw data

what was measured

split

train / val / test

transform

fit learned state
on train only

batch

x, y, mask, ...

fit only on train · select on validation · report generalization on test

Tensor Semantics

Invariant: shape · dtype · device

Shape is a type that Python does not check.

[B, T, D]

batch · token · feature

float32

compute / range / precision

cuda:0

placement / transfer

Check at boundaries:

input → module → loss → metric

finite? expected?

Batching

Variable-length batching = padded values + a valid-position mask

examples

padded values

valid mask

$[T_i, D]$



$[B, T_{max}, D]$



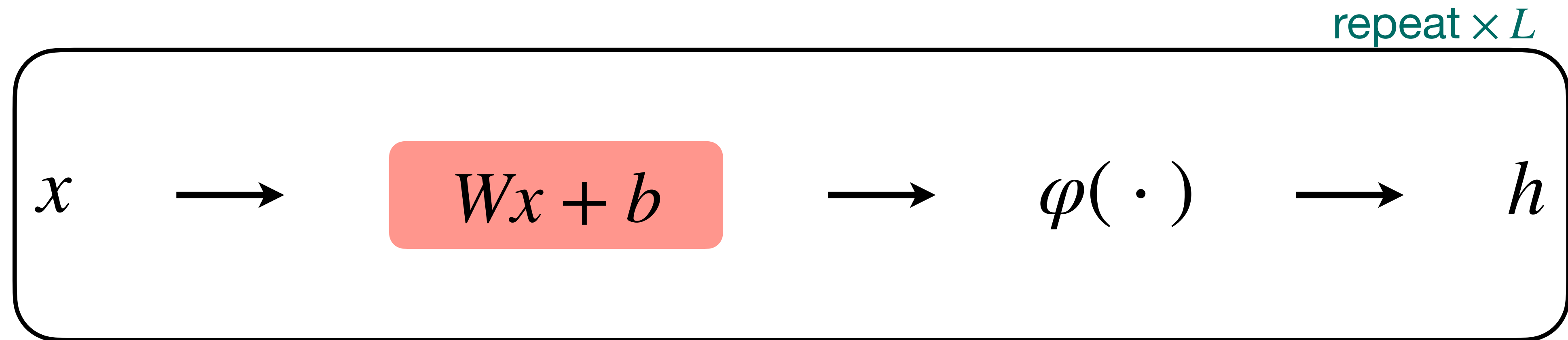
$[B, T_{max}]$

Loss mask: valid tokens · ignore_index / weights

Padding changes storage; the mask restores the semantics of the original sequence lengths

Basic recall

Linear layer + nonlinearity + depth



$$h = \varphi(Wx + b)$$

Without $\varphi(\cdot)$, stacked $Wx + b$ layers collapse to one affine map.

Trainability

Depth works only if signals and gradients stay in range

Initialization

Xavier / He

keep signal variance
in range at the start

Activation

ReLU / GELU

add nonlinearity and
shape gradient
propagation

Residual path

$x + F(x)$

provide short signal and
gradients routes

Normalization

**Batchnorm /
LayerNorm**

control activation
scale and statistics

Initialization, residuals and normalization primarily solve trainability — not generalization.

Objective

The head, target, loss, and inference rule must describe the same task

Task	Output	Target	Loss	Inference
Regression	$\hat{y} \in \mathbb{R}$	continuous value	MSE / MAE	identity
Multiclass	C logits	class index	CrossEntropy	argmax
Multilabel	C logits	C-bit vector	BCEWithLogits	sigmoid + threshold
Autoregressive	T x V logits	next token	token CE	sampling

A backbone builds a representation; the head maps them to task-specific outputs.

Objective

Logits $\rightarrow$ loss: preserve numerical stability

WRONG: $\text{softmax}(\text{logits}) \rightarrow \log() \rightarrow \text{NLL}$

RIGHT: $\text{CrossEntropyLoss}(\text{logits}, \text{target}) = \text{NLLLoss}(\log_softmax(\text{logits}), \text{target})$

BCEWithLogitsLoss fuses sigmoid + BCE for the same reason.

Log-sum-exp(): uses the log-sum-exp trick.

Reduction

Reduction defines the scale and weighting of the experiment

$$L = \frac{1}{B} \sum_i l_i$$

mean per examples

stable when batch
size changes

$$L = \sum_i l_i$$

sum over examples

scale changes with
the number of terms

$$L = \frac{\sum_i m_i w_i l_i}{\sum_i m_i w_i}$$

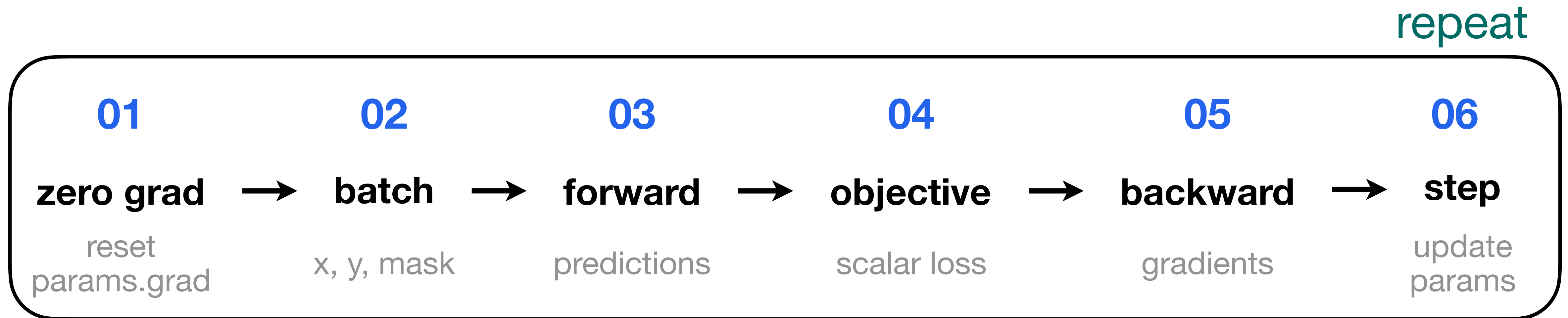
masked / weighted mean

denominator defines
the valid units

Across microbatches reproduce the intended global numerator and denominator; divide once before update.

Training step

One training step underlies every model



At every boundary, check shape · dtype · scale · model mode.

With accumulation repeat batch → forward → objective → backward K times before one step.

Backpropagation

The chain rule links local derivatives

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial \theta}$$

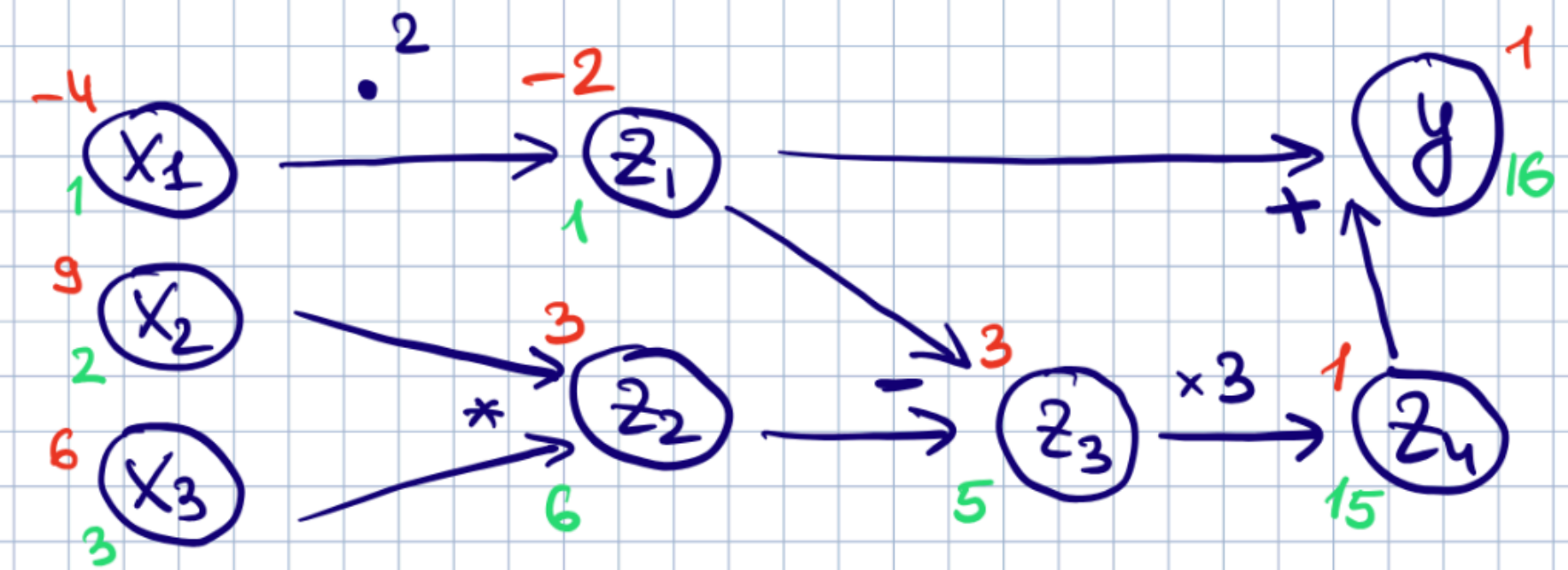
Each operation uses its saved forward values to transform the incoming gradient into gradients for its inputs.

A parameter gradient sums contributions from every path involving it.

Gradients accumulate until they are explicitly reset.

$$y = f(x_1, x_2, x_3) = x_1^2 + 3(x_2 \cdot x_3 - x_1^2)$$

Goal: compute $\frac{dy}{dx_i}$ $x_1=1, x_2=2, x_3=3$



Forward pass
(node's function values)

- $z_1 = x_1^2 = 1$
- $z_2 = x_2 \cdot x_3 = 6$
- $z_3 = z_2 - z_1 = 5$
- $z_4 = 3 \cdot z_3 = 15$
- $y = z_1 + z_4 = 16$

Backward pass
(node's derivative values)

- $\frac{dy}{dy} = 1$ (dummy step)
- $\frac{dy}{dz_4} = \frac{dy}{dy} \cdot \frac{dy}{dz_4} = 1 \cdot 1 = 1$
- $\frac{dy}{dz_3} = \frac{dy}{dz_4} \cdot \frac{dz_4}{dz_3} = 1 \cdot 3 = 3$
- $\frac{dy}{dz_2} = \frac{dy}{dz_3} \cdot \frac{dz_3}{dz_2} = 3 \cdot 1 = 3$
- $\frac{dy}{dx_2} = \frac{dy}{dz_2} \cdot \frac{dz_2}{dx_2} = 3 \cdot x_3 = 9$
- $\frac{dy}{dx_3} = \frac{dy}{dz_2} \cdot \frac{dz_2}{dx_3} = 3 \cdot x_2 = 6$

Optimization

The optimizer maps the current gradient and state to an update

$$\theta_{t+1} = \theta_t - \eta \cdot U(g_t, s_t)$$

η — learning rate

s_t — optimizer state

SGD

uses the gradients directly

Momentum

maintains a velocity/
first-moment EMA

RMSprop

scales coordinates by an
EMA of squared gradients

Normalization

uses bias-corrected first-
and second-moment EMAs

The optimizer changes the update rule — not the objective.

Optimization

Learning rate sets step scale; effective batch size affects gradient noise

too low lr

progress, but too slow

stable lr range

fast + stable

too high lr

oscillation / divergence

Effective batch size changes gradient-estimate variance, memory use and — with BatchNorm — the layer statistics.

Stabilization $\neq$ regularization

Different mechanisms solve different problems

Training stabilization

- Initialization — preserve signal scale
- Normalization — control activations
- Residual paths — improve gradient flow
- Gradient clipping — limit the current gradient norm

Generalization control

- Weight decay — constrain parameter norm
- Dropout — stochastic feature removal
- Augmentation — encode desired invariance
- Early stopping — select a validation checkpoint

BatchNorm may regularize, but that is not its primary contract; LayerNorm is not a regularizer.

Evaluation & inference

Module mode, autograd and parameter updates are separate controls

Control	What it changes	Typical use
<code>model.train() / model.eval()</code>	Dropout and BatchNorm behavior	gradient tracking
<code>no_grad() / inference_mode()</code>	grad construction and memory	module mode
<code>param.requires_grad</code>	whether autograd accumulates new <code>.grad</code>	optimizer membership or module mode
<code>optimizer param_groups</code>	eligibility for <code>optimizer.step()</code>	graph construction or module mode

Validation normally requires both `model.eval()` and `torch.inference_mode()`.

Fine-tuning

Reuse a pretrained backbone; choose which parameters to update

$$\theta_0 = \theta_{\text{pretrained}} \longrightarrow \theta^* = \operatorname{argmin} L_{\text{target}}$$

The training loop is unchanged: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update.

Mode	Init	Backbone	Head	What it tests
Scratch	random	train	train	baseline
Linear probe	pretrained	frozen	train	representation
Partial FT	pretrained	partially trainable	train	adaptability
Full FT	pretrained	train	train	init + adaptation

Before trusting a result

Eight questions define the experiment

- 01** What is one example, and how was the split made?
- 02** What are the shape, dtype, device and mask?
- 03** What exactly do the backbone and head output?
- 04** Do output, target and loss describe the same task?
- 05** How is the objective reduced to a scalar?
- 06** Where does the gradient flow, and what is trainable?
- 07** What are the update rule, LR and effective batch?
- 08** How do validation and inference differ from training?

Questions and cases

Diagnose the broken contract before proposing the fix.

01

Hypothesis

What exactly is broken?

02

Mechanism

Which DL contract explains it?

03

Test

What observation would decide?

Each case appears twice: discuss first, then reveal the answer.

Question 01

Data · Splits · Preprocessing

Given

Normalization statistics are fitted on the full dataset before the train/validation split. Validation is unusually strong.

Case

Why is this validation score not trustworthy?

Guiding
questions

Answer

Question 01

Data · Splits · Preprocessing

Given

Normalization statistics are fitted on the full dataset before the train/validation split. Validation is unusually strong.

Case

Why is this validation score not trustworthy?

Guiding questions

- Which object learned something from the data?
- Which split is allowed to fit it?
- What comparison would expose the leakage?

Answer

Question 01

Data · Splits · Preprocessing

Given

Normalization statistics are fitted on the full dataset before the train/validation split. Validation is unusually strong.

Case

Why is this validation score not trustworthy?

Guiding questions

- Which object learned something from the data?
- Which split is allowed to fit it?
- What comparison would expose the leakage?

Answer

- **Diagnosis** — stateful preprocessing leaked validation information into training.
- **Mechanism** — learned transforms are part of the fitted model even without gradients.
- **Fix / check** — split first; fit on train only; freeze for validation and test; compare with the clean pipeline.

Question 02

Tensor semantics · Broadcasting

Given

Predictions have shape $[B, 1]$, targets have shape $[B]$, and `MSELoss` returns a scalar without raising an exception.

Case

How can the loss be mathematically wrong while the code still runs?

Guiding
questions

Answer

Question 02

Tensor semantics · Broadcasting

Given

Predictions have shape $[B, 1]$, targets have shape $[B]$, and `MSELoss` returns a scalar without raising an exception.

Case

How can the loss be mathematically wrong while the code still runs?

Guiding questions

- What shape does broadcasting produce?
- Are predictions paired with targets one-to-one?
- Which invariant should be asserted at this boundary?

Answer

Question 02

Tensor semantics · Broadcasting

Given

Predictions have shape $[B, 1]$, targets have shape $[B]$, and `MSELoss` returns a scalar without raising an exception.

Case

How can the loss be mathematically wrong while the code still runs?

Guiding questions

- What shape does broadcasting produce?
- Are predictions paired with targets one-to-one?
- Which invariant should be asserted at this boundary?

Answer

- **Diagnosis** — broadcasting creates pairwise errors instead of aligned errors.
- **Mechanism** — $[B, 1]$ and $[B]$ expand to $[B, B]$; valid syntax changed the semantics.
- **Fix / check** — make shapes exactly equal and assert them at the model-to-loss boundary.

Question 03

Batching · Padding · Masks

Given

Variable-length sequences are padded to the longest item in each batch, then every position is mean-pooled without a mask.

Case

Why can one example depend on which neighbors share its batch?

Guiding
questions

Answer

Question 03

Batching · Padding · Masks

Given

Variable-length sequences are padded to the longest item in each batch, then every position is mean-pooled without a mask.

Case

Why can one example depend on which neighbors share its batch?

Guiding questions

- Who determines the padded length?
- Where can PAD affect valid positions?
- What should be used as the denominator?

Answer

Question 03

Batching · Padding · Masks

Given

Variable-length sequences are padded to the longest item in each batch, then every position is mean-pooled without a mask.

Case

Why can one example depend on which neighbors share its batch?

Guiding questions

- Who determines the padded length?
- Where can PAD affect valid positions?
- What should be used as the denominator?

Answer

- **Diagnosis** — PAD tokens enter both interactions and the final reduction.
- **Mechanism** — batch composition changes PAD count; transformed PAD values need not stay neutral.
- **Fix / check** — mask every relevant interaction and use a masked reduction; test the item alone and in a mixed batch.

Question 04

Model · Nonlinearity · Trainability

Given

A 12-layer MLP stacks affine Linear layers with biases but uses no nonlinear activations.

Case

What expressive power did the extra depth actually add?

Guiding
questions

Answer

Question 04

Model · Nonlinearity · Trainability

Given

A 12-layer MLP stacks affine Linear layers with biases but uses no nonlinear activations.

Case

What expressive power did the extra depth actually add?

Guiding questions

- What is a composition of affine maps?
- Which decision boundaries can it represent?
- What changes after adding nonlinearities?

Answer

Question 04

Model · Nonlinearity · Trainability

Given

A 12-layer MLP stacks affine Linear layers with biases but uses no nonlinear activations.

Case

What expressive power did the extra depth actually add?

Guiding questions

- What is a composition of affine maps?
- Which decision boundaries can it represent?
- What changes after adding nonlinearities?

Answer

- **Diagnosis** — the stack represents one affine map, possibly with an extra rank bottleneck.
- **Mechanism** — depth changes parameterization, but affine composition cannot create nonlinear boundaries.
- **Fix / check** — add nonlinearities; then verify activation and gradient scales with suitable initialization or residuals.

Question 05

Head · Target · Loss contract

Given

A single-label C-class classifier outputs C logits. Targets are one-hot float vectors, and training uses BCEWithLogitsLoss.

Case

What task is this code actually optimizing?

Guiding
questions

Answer

Question 05

Head · Target · Loss contract

Given

A single-label C-class classifier outputs C logits. Targets are one-hot float vectors, and training uses BCEWithLogitsLoss.

Case

What task is this code actually optimizing?

Guiding questions

- Are classes mutually exclusive or independent?
- How is the target encoded?
- Which inference rule matches the objective?

Answer

Question 05

Head · Target · Loss contract

Given

A single-label C-class classifier outputs C logits. Targets are one-hot float vectors, and training uses BCEWithLogitsLoss.

Case

What task is this code actually optimizing?

Guiding questions

- Are classes mutually exclusive or independent?
- How is the target encoded?
- Which inference rule matches the objective?

Answer

- **Diagnosis** — the code optimizes multilabel classification, not multiclass classification.
- **Mechanism** — BCE treats C labels independently; multiclass cross-entropy couples the logits.
- **Fix / check** — for ordinary single-label training, use class indices with CrossEntropyLoss and argmax.

Question 06

Numerical stability · Logits to loss

Given

The code computes `softmax(logits).log()` before `NLLLoss`. Under mixed precision, some batches produce `inf` or `NaN`.

Case

Why is mathematically equivalent code numerically unstable?

Guiding
questions

Answer

Question 06

Numerical stability · Logits to loss

Given

The code computes `softmax(logits).log()` before `NLLLoss`. Under mixed precision, some batches produce `inf` or `NaN`.

Case

Why is mathematically equivalent code numerically unstable?

Guiding questions

- What happens when a probability underflows to zero?
- What input does `CrossEntropyLoss` expect?
- Which stable identity avoids explicit probabilities?

Answer

Question 06

Numerical stability · Logits to loss

Given

The code computes `softmax(logits).log()` before `NLLLoss`. Under mixed precision, some batches produce `inf` or `NaN`.

Case

Why is mathematically equivalent code numerically unstable?

Guiding questions

- What happens when a probability underflows to zero?
- What input does `CrossEntropyLoss` expect?
- Which stable identity avoids explicit probabilities?

Answer

- **Diagnosis** — the explicit probability-to-log path underflows.
- **Mechanism** — tiny probabilities round to zero, then $\log(0)$ becomes $-\infty$.
- **Fix / check** — pass logits to `CrossEntropyLoss`; test extreme logits and assert finite loss and gradients.

Question 07

Reduction · Gradient scale

Given

Token losses are summed over valid positions. Sequence lengths vary across batches, while the learning rate stays fixed.

Case

Why does the update scale change with batch composition?

Guiding
questions

Answer

Question 07

Reduction · Gradient scale

Given

Token losses are summed over valid positions. Sequence lengths vary across batches, while the learning rate stays fixed.

Case

Why does the update scale change with batch composition?

Guiding questions

- Is the reduction a sum or a mean?
- What is the unit of one training example?
- Which denominator should remain explicit?

Answer

Question 07

Reduction · Gradient scale

Given

Token losses are summed over valid positions. Sequence lengths vary across batches, while the learning rate stays fixed.

Case

Why does the update scale change with batch composition?

Guiding questions

- Is the reduction a sum or a mean?
- What is the unit of one training example?
- Which denominator should remain explicit?

Answer

- **Diagnosis** — the loss scale follows the number of valid tokens.
- **Mechanism** — a sum makes gradient magnitude proportional to the number of terms.
- **Fix / check** — normalize by the target unit, aggregate the denominator across micro-batches, and log it.

Question 08

Training step · Gradient accumulation

Given

`zero_grad()` is called once per epoch, while every batch runs `backward()` followed by `optimizer.step()`. The author calls this a larger batch.

Case

Why is this not valid gradient accumulation?

Guiding
questions

Answer

Question 08

Training step · Gradient accumulation

Given

`zero_grad()` is called once per epoch, while every batch runs `backward()` followed by `optimizer.step()`. The author calls this a larger batch.

Case

Why is this not valid gradient accumulation?

Guiding questions

- Does `optimizer.step()` clear `.grad`?
- Are all gradients computed at the same parameters?
- What must remain fixed across microbatches?

Answer

Question 08

Training step · Gradient accumulation

Given

`zero_grad()` is called once per epoch, while every batch runs `backward()` followed by `optimizer.step()`. The author calls this a larger batch.

Case

Why is this not valid gradient accumulation?

Guiding questions

- Does `optimizer.step()` clear `.grad`?
- Are all gradients computed at the same parameters?
- What must remain fixed across microbatches?

Answer

- **Diagnosis** — stale gradients are reused after the parameters move.
- **Mechanism** — `step()` does not clear `.grad`; true accumulation keeps θ fixed for K microbatches.
- **Fix / check** — zero grad once, run K normalized backwards, step optimizer; verify against a full batch.

Question 09

Backpropagation · Graph paths

Given

An auxiliary loss is computed from `features.detach()`. Total loss decreases, but this loss cannot influence the backbone.

Case

Where did the gradient path disappear?

Guiding
questions

Answer

Question 09

Backpropagation · Graph paths

Given

An auxiliary loss is computed from `features.detach()`. Total loss decreases, but this loss cannot influence the backbone.

Case

Where did the gradient path disappear?

Guiding questions

- Does `detach()` change the forward value?
- Which parameters are reachable from this loss?
- How can we verify each loss branch separately?

Answer

Question 09

Backpropagation · Graph paths

Given

An auxiliary loss is computed from `features.detach()`. Total loss decreases, but this loss cannot influence the backbone.

Case

Where did the gradient path disappear?

Guiding questions

- Does `detach()` change the forward value?
- Which parameters are reachable from this loss?
- How can we verify each loss branch separately?

Answer

- **Diagnosis** — `detach()` cut the auxiliary path to the backbone.
- **Mechanism** — backprop follows graph edges, not conceptual ownership of a module.
- **Fix / check** — remove unintended `detach`; use `autograd.grad` or isolate each branch and inspect backbone gradients.

Question 10

Optimizer membership · Fine-tuning

Given

The backbone is unfrozen with `requires_grad_(True)` after the optimizer was created from `head.parameters()`. Backbone gradients are nonzero, but its weights never change.

Case

Why does an unfrozen module still fail to update?

Guiding
questions

Answer

Question 10

Optimizer membership · Fine-tuning

Given

The backbone is unfrozen with `requires_grad_(True)` after the optimizer was created from `head.parameters()`. Backbone gradients are nonzero, but its weights never change.

Case

Why does an unfrozen module still fail to update?

Guiding questions

- Does `requires_grad` modify optimizer groups?
- Which parameters does `step()` actually update?
- What state must be added or rebuilt?

Answer

Question 10

Optimizer membership · Fine-tuning

Given

The backbone is unfrozen with `requires_grad_(True)` after the optimizer was created from `head.parameters()`. Backbone gradients are nonzero, but its weights never change.

Case

Why does an unfrozen module still fail to update?

Guiding questions

- Does `requires_grad` modify optimizer groups?
- Which parameters does `step()` actually update?
- What state must be added or rebuilt?

Answer

- **Diagnosis** — the optimizer has no references to backbone parameters.
- **Mechanism** — gradient computation and optimizer membership are independent controls.
- **Fix / check** — add a group to optimizer with explicit LR/scheduler handling, or rebuild and manage optimizer state.

Question 11

Learning rate · Effective batch · BatchNorm

Given

The physical batch drops from 32 to 2. Gradient accumulation keeps the optimizer batch at 32, but a BatchNorm model becomes unstable.

Case

Why did accumulation not restore the original behavior?

Guiding
questions

Answer

Question 11

Learning rate · Effective batch · BatchNorm

Given

The physical batch drops from 32 to 2. Gradient accumulation keeps the optimizer batch at 32, but a BatchNorm model becomes unstable.

Case

Why did accumulation not restore the original behavior?

Guiding questions

- Which batch does BatchNorm see?
- Are forward statistics accumulated across microbatches?
- Which batch notions remained different?

Answer

Question 11

Learning rate · Effective batch · BatchNorm

Given

The physical batch drops from 32 to 2. Gradient accumulation keeps the optimizer batch at 32, but a BatchNorm model becomes unstable.

Case

Why did accumulation not restore the original behavior?

Guiding questions

- Which batch does BatchNorm see?
- Are forward statistics accumulated across microbatches?
- Which batch notions remained different?

Answer

- **Diagnosis** — the optimizer batch was restored, but the normalization batch was stayed at 2.
- **Mechanism** — accumulation combines gradients, but each forward compute BN statistics on one microbatch.
- **Fix / check** — use a larger microbatch or frozen pretrained BN.

Question 12

Stabilization · Regularization

Given

Training loss diverges and gradient norms explode. The proposed first fix is stronger dropout and heavier data augmentation.

Case

Why should we test a different hypothesis first?

Guiding
questions

Answer

Question 12

Stabilization · Regularization

Given

Training loss diverges and gradient norms explode. The proposed first fix is stronger dropout and heavier data augmentation.

Case

Why should we test a different hypothesis first?

Guiding questions

- Is this optimization failure or a generalization gap?
- Which mechanisms directly bound signal or update scale?
- What minimal test should the model pass first?

Answer

Question 12

Stabilization · Regularization

Given

Training loss diverges and gradient norms explode. The proposed first fix is stronger dropout and heavier data augmentation.

Case

Why should we test a different hypothesis first?

Guiding questions

- Is this optimization failure or a generalization gap?
- Which mechanisms directly bound signal or update scale?
- What minimal test should the model pass first?

Answer

- **Diagnosis** — the evidence points to stability failure, not overfitting.
- **Mechanism** — dropout and augmentation target generalization and may add more noise.
- **Fix / check** — inspect data/loss scale, AMP, LR, activations, and gradients; stabilize the run and overfit one batch first.

Question 13

Evaluation · Inference · Fine-tuning

Given

A pretrained backbone is frozen, but the whole model stays in `train()` during head fine-tuning and validation under `no_grad()`.

Case

Why is a frozen backbone still changing the evaluated function?

Guiding
questions

Answer

Question 13

Evaluation · Inference · Fine-tuning

Given

A pretrained backbone is frozen, but the whole model stays in `train()` during head fine-tuning and validation under `no_grad()`.

Case

Why is a frozen backbone still changing the evaluated function?

Guiding questions

- What does `no_grad()` control?
- What does `train()` versus `eval()` control?
- Which state is not a Parameter?

Answer

Question 13

Evaluation · Inference · Fine-tuning

Given

A pretrained backbone is frozen, but the whole model stays in `train()` during head fine-tuning and validation under `no_grad()`.

Case

Why is a frozen backbone still changing the evaluated function?

Guiding questions

- What does `no_grad()` control?
- What does `train()` versus `eval()` control?
- Which state is not a Parameter?

Answer

- **Diagnosis** — frozen weights do not imply frozen module behavior.
- **Mechanism** — `train()` keeps Dropout stochastic and updates BatchNorm buffers; `no_grad` only disables graph building.
- **Fix / check** — keep frozen backbone in `eval()`, validate in `inference_mode()`, and verify buffers/repeatability.



Q&A

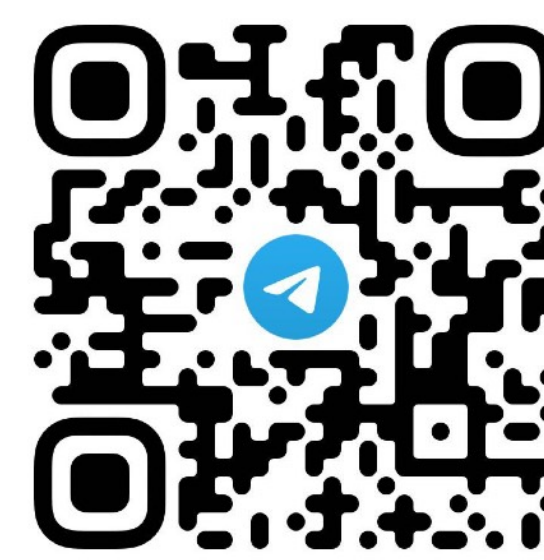
My LinkedIn



Course Github repo



BayesLab seminar



Thank you for your attention!

